

Image Watermarking Tool

Deep Dive: how every part of it works

Explainer document for the project owner

This document explains, in full, exactly how the `image-watermarking-tool` project works. It is a working reference for the owner, not published website content. Everything stated here is drawn directly from the project’s source (`main.py`) and its `README.md`; nothing is invented. Any place where something had to be inferred rather than read directly is marked `[inferred]`.

Contents

1	What this project is	3
1.1	Technology building blocks (defined before use)	3
2	High-level architecture	4
3	Directory layout	5
4	The image-processing functions	5
4.1	<code>compute_watermark_box(watermark, max_width, max_height)</code>	5
4.2	<code>apply_opacity(watermark, opacity_percent)</code>	5
4.3	<code>compute_position(base_w, base_h, wm_w, wm_h, preset, offset_x_pct, offset_y_pct)</code>	5
4.4	<code>paste_watermark_at(base_image, watermark, x, y)</code> — the shared compositing step	6
5	The bug that was found and fixed: Pillow’s <code>paste()</code> without a mask	6
5.1	What the original, broken code did	6
5.2	Why this bug is easy to miss	6
5.3	The fix	6
5.4	The verification: an actual pixel-level before/after comparison	7
6	Data flow: live preview vs. final save	7
7	Debouncing: why the preview doesn’t redraw on every pixel of drag	8
8	The GUI layout, widget by widget	8
9	The full user actions	9
9.1	<code>upload_watermark()</code>	9
9.2	<code>upload_photos()</code>	9
9.3	<code>apply_and_save()</code>	9
9.4	<code>save_watermarked_file(image, source_path, location, file_format)</code>	10
10	Documented challenges (from the README) and how each was handled	10
11	Testing and verification methodology	10
11.1	What is explicitly <i>not</i> verified	11

12 What the author says they'd do differently	11
13 Glossary (quick reference)	11

1 What this project is

The Image Watermarking Tool is a small desktop application with a graphical window (not a web page, not a command-line script) that lets a person stamp a logo or watermark image onto a batch of photographs. A person picks a watermark image, picks one or more photos, adjusts a handful of sliders (size, transparency, position), watches a live preview update, and then clicks one button to save watermarked copies of every selected photo.

Why it exists (from the README): batch-watermarking photos is a repetitive task most people either do by hand in a full image editor (like Photoshop or GIMP) or skip entirely. This tool automates exactly that one job and nothing else: take a watermark, size and fade it, apply it to many photos in one pass, and never touch the original files.

The entire application is one Python file, `main.py`, about 390 lines long. There is no server, no database, no network calls, and no configuration files to fill in — everything the program needs is either hard-coded as a constant or entered live through the GUI.

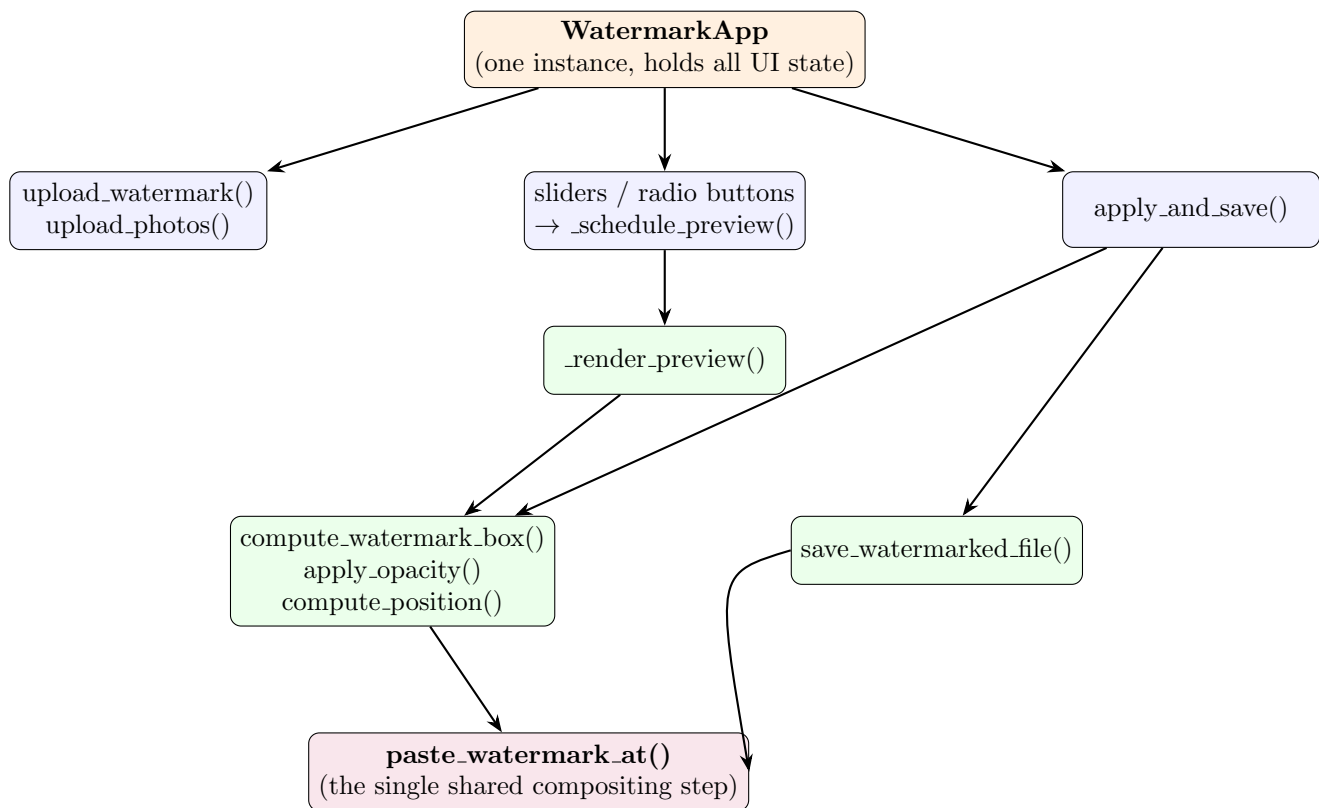
1.1 Technology building blocks (defined before use)

- **Python** — the programming language the whole project is written in. Python is read top to bottom by an interpreter, no separate compile step.
- **GUI (Graphical User Interface)** — a program interface made of visible windows, buttons, sliders, and text, as opposed to a program you only type commands into.
- **tkinter** — Python’s built-in library for building GUI windows. It ships with most desktop Python installations (the README notes that on minimal Linux installs it may need a separate `apt install python3-tk`). `tkinter` provides widgets: **Button**, **Label**, **Scale** (a slider), **Radiobutton**, **Canvas** (a drawing surface), and layout containers such as **Frame** and **LabelFrame**.
- **ttk** — a themed variant of `tkinter`’s widgets (nicer-looking buttons, sliders, frames) that this project uses alongside plain `tkinter` widgets.
- **Pillow (the PIL package)** — the Python library used to open, resize, blend, and save image files (JPEG, PNG). Pillow represents an open image as an **Image** object with a **mode** (how its pixels are stored) and a size.
- **RGB / RGBA** — pixel color modes. **RGB** stores three numbers per pixel: red, green, blue, each 0–255. **RGBA** adds a fourth number, **alpha**, which encodes opacity: 0 is fully transparent, 255 is fully opaque. JPEG files cannot store an alpha channel; PNG files can.
- **Alpha compositing / alpha blending** — the process of combining a semi-transparent foreground image with a background image by mixing their color channels in proportion to the foreground’s alpha value at each pixel. This is the core operation this whole tool depends on, and it is the source of the bug described in Section 5.
- **Mask (in Pillow’s paste call)** — an optional third argument to `Image.paste()` that tells Pillow which pixels of the pasted image should show through and by how much, using the mask image’s own values (commonly its alpha channel) as blending weights.
- **Bounding box resize** — resizing an image so it fits within a maximum width and height while keeping its original width-to-height ratio (**aspect ratio**), so it does not look stretched or squashed.

- **Debounce** — delaying an action until a rapid burst of trigger events (like dragging a slider) has paused, so the action runs once instead of dozens of times per second.
- `pathlib.Path` — Python’s object-oriented way of working with filesystem paths; it automatically uses the correct path separator (`/` on Linux/macOS, `\` on Windows) instead of a program manually splitting strings on `/`.

2 High-level architecture

Everything is one class, `WatermarkApp`, plus a handful of free (module-level) functions that do the actual image math. The class holds **state** (the things the program needs to remember between user actions: the loaded watermark, the list of selected photo paths, and every slider’s current value) as **instance attributes** — values attached to the specific `WatermarkApp` object rather than shared globally. The README is explicit that this replaced an earlier version of the code that used plain module-level global variables instead; keeping state on the instance is what allows the class to be constructed, driven, and inspected programmatically in tests (see Section 11), since each instance is independent.



The critical design point, stated explicitly in the README, is that **the live preview and the final saved file are produced by the exact same compositing function, `paste_watermark_at()`**. There is only one code path that blends a watermark onto a photo; the preview just runs it on a shrunk-down copy first. This matters because it removes an entire class of bug where “what you see is not what you get” — Section 6 covers this in more detail.

3 Directory layout

```
image-watermarking-tool/  
  main.py           all application code  
  README.md         usage, design notes, verification log  
  requirements.txt   Pillow>=10.0,<13.0  
  LICENSE           PolyForm Strict 1.0.0  
  .gitignore
```

There is deliberately no `src/` tree, no package structure, and no `.env` file: the README notes explicitly that “no environment variables are read anywhere in this project.” For a single-window desktop tool of this size, one flat file is proportionate; the README itself acknowledges in “What I’d do differently” that the project would benefit from an automated test suite, but does not propose splitting the file further.

4 The image-processing functions

These are the module-level functions (not attached to the `WatermarkApp` class) that do the actual pixel work. Each is a small, pure-ish function: given the same inputs, it produces the same outputs, which is what makes it possible to reuse them identically in both the preview path and the save path.

4.1 `compute_watermark_box(watermark, max_width, max_height)`

Takes a copy of the watermark image and calls Pillow’s `Image.thumbnail()` on it with a `(max_width, max_height)` box and **LANCZOS** resampling (a high-quality image resizing algorithm that reduces blurring/aliasing compared to simpler methods). **Important behavior:** `thumbnail()` mutates its target image **in place** and always preserves aspect ratio, treating the given size as an upper bound rather than an exact target — this is explicitly called out in the README’s “What I learned” section as the reason the width/height sliders act like a bounding box rather than forcing an exact size.

4.2 `apply_opacity(watermark, opacity_percent)`

Forces the watermark to **RGBA** mode (adding an alpha channel if it doesn’t have one), splits it into its four channels (`r`, `g`, `b`, `a`) with `Image.split()`, and rewrites the alpha channel by scaling every alpha value by `opacity_percent / 100.0` using ImageChops-free per-pixel mapping (`a.point(lambda v: int(v * scale))`). A 50% opacity setting, for example, halves every alpha value in the image; a fully-opaque pixel (255) becomes roughly 127.

4.3 `compute_position(base_w, base_h, wm_w, wm_h, preset, offset_x_pct, offset_y_pct)`

Looks up a **preset** (one of nine named positions) in the `POSITION_PRESETS` dictionary of `lambda` functions, which returns a base top-left (`x`, `y`) pixel coordinate for the watermark given the photo and watermark dimensions. It then nudges that position by a percentage-of-image-size offset (`offset_x_pct`, `offset_y_pct`, each allowed from -20% to $+20\%$ in the UI) so the user can fine-tune away from the nine fixed spots without needing free-form dragging.

TL	TC	TR
ML	C	MR
BL	BC	BR

3×3 preset grid, plus X/Y fine-offset sliders (±20% of photo size) not shown

4.4 `paste_watermark_at(base_image, watermark, x, y)` — the shared compositing step

This is the single function both the live preview and the final save call. If the watermark is in RGBA mode, the base photo is converted to RGBA too and Pillow’s `paste()` is called **with the watermark passed twice**: once as the image to paste, once again as the **mask** argument (`result.paste(watermark, (x, y), watermark)`). Passing an image’s own alpha channel as its own mask is what makes Pillow perform a genuine alpha blend instead of a flat overwrite. If the watermark has no alpha channel at all, it falls back to a plain paste with no mask, since there is nothing to blend against in that case.

5 The bug that was found and fixed: Pillow’s `paste()` without a mask

This is the single most important technical story in this project, and it is documented in the README with concrete pixel evidence, reproduced here in full.

5.1 What the original, broken code did

The original `add_watermark_to_images()` function called:

```
uploaded_image.paste(watermark, (x, y))
```

with **no third argument**. When you call Pillow’s `Image.paste()` with only a position and no mask, Pillow does not blend anything — it literally copies the pasted image’s pixel values (including its own alpha channel, if it has one) directly on top of the destination’s pixels, discarding whatever was there before at every covered pixel.

5.2 Why this bug is easy to miss

The watermark image still had an opacity value baked into its own alpha channel (from `apply_opacity()`), and that alpha channel survived the paste. So when you opened the saved file in an image viewer that respects alpha, the watermark region *looked* translucent — the file format was technically storing “50% opacity” as a number. But the *color* underneath was never actually mixed with the photo; the photo’s original pixel colors in that region were gone, overwritten outright. The translucency was cosmetic metadata, not a real blend.

5.3 The fix

In `paste_watermark_at()` (Section 4.4), the fix is to pass the watermark image a second time, as the mask:

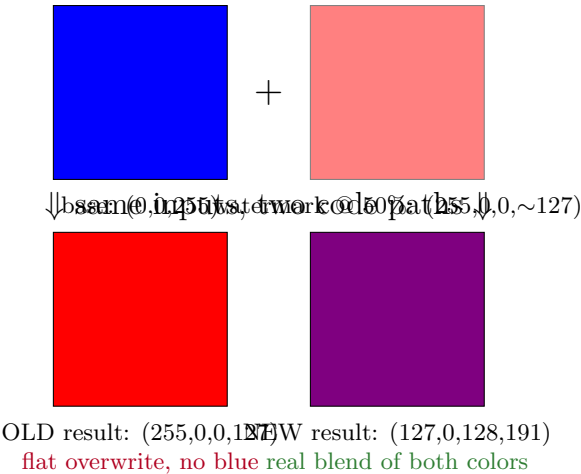
```
base.paste(watermark, (x, y), watermark)
```

Using an image’s own alpha channel as its own paste mask tells Pillow: ”at each pixel, blend the incoming color and the existing color, weighted by how opaque this alpha value says the incoming pixel is.” That is a genuine weighted average of two colors, not a copy.

5.4 The verification: an actual pixel-level before/after comparison

The README documents a synthetic test built specifically to prove the difference: a solid **blue** base ”photo” ((0, 0, 255) in RGB — pure blue, no red, no green) and a solid **red** watermark ((255, 0, 0)) faded to 50% opacity via `apply_opacity()`, which scales the alpha to approximately 127/255. Both the old (no-mask) and new (mask) code paths were run on the same two images, and the pixel at the same watermark-covered location was read back from each result:

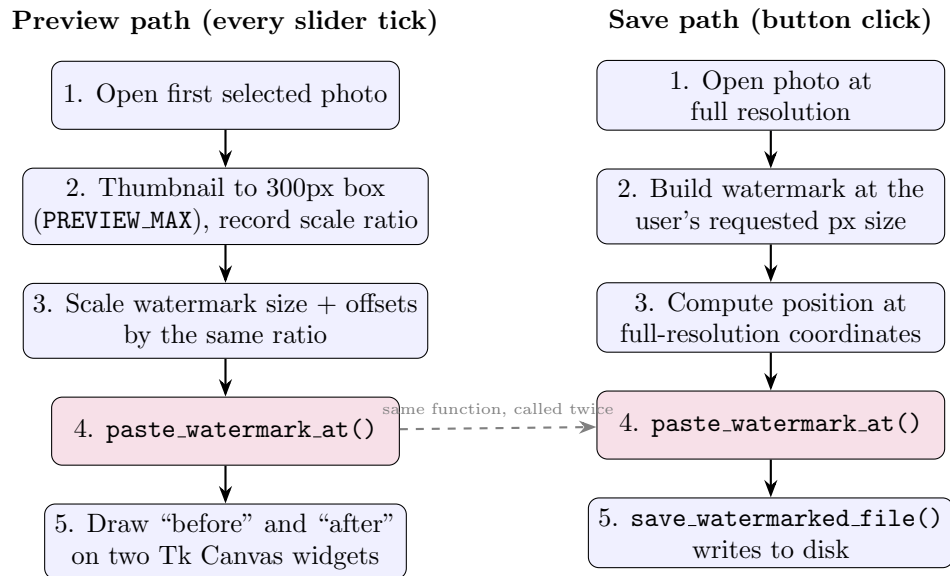
Code path	Resulting pixel (RGBA)	Interpretation
OLD <code>paste()</code> , no mask	(255, 0, 0, 127)	Flat red overwrite; blue channel is exactly 0 — the base photo’s blue contributed nothing at all.
NEW <code>paste_watermark_at()</code> , mask = watermark	(127, 0, 128, 191)	A genuine mixed color: red pulled down from 255 to 127, blue pulled up from 0 to 128 — both source images visibly contributed.



The old path’s result keeps the base’s blue channel at exactly zero — mathematical proof that no blending occurred, only a flat overwrite with the stored alpha value carried along for show. The new path’s blue channel rises from 0 to 128 and its red channel is pulled down from 255 to 127: a real weighted mix of the watermark’s red and the photo’s blue, consistent with alpha compositing math. This exact test (script and output) is reproduced again in the README’s Verification section, run a second time as part of confirming the shipped code, with identical results.

6 Data flow: live preview vs. final save

A key design goal stated in the README is that ”what’s shown on screen before saving is what actually gets written to disk, not an approximation.” Both flows below share step 4, `paste_watermark_at()`; the only difference is which image size they operate on and what happens to the result afterward.

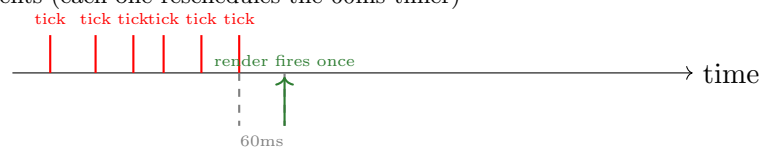


Scaling the watermark’s size and offset by the same ratio the base photo was shrunk by (step 3 on the left) is what keeps the small preview visually representative of the full-resolution result, without ever compositing a full-size image just to show a thumbnail — the README calls this out explicitly as the fix for keeping live preview fast regardless of source photo size (Section 10).

7 Debouncing: why the preview doesn’t redraw on every pixel of drag

Every slider and radio button is wired to call `_schedule_preview()` rather than `_render_preview()` directly. `_schedule_preview()` cancels any pending redraw (`root.after_cancel`) and schedules a new one 60 milliseconds in the future (`root.after(60, self._render_preview)`). If the user is still dragging when that timer fires, the timer gets cancelled and rescheduled again. The practical effect: dragging a slider across its full range does not trigger dozens of redraws per second, only one shortly after the user stops moving it. This pattern is called a **debounce**.

rapid slider drag events (each one reschedules the 60ms timer)



8 The GUI layout, widget by widget

`_build_layout()` creates a title label, then splits the window into a left **controls** column and a right **preview** column using `ttk.Frame` with `pack(side='left', ...)`, plus a status label pinned to the bottom of the window. **Packing** is one of tkinter’s layout systems: it stacks widgets against a side of their parent (`top`, `bottom`, `left`, `right`) in the order they’re added.

`_build_controls()` lays out four numbered `LabelFrame` groups (a frame widget with a visible border and title):

- **1. Load images** — “Upload Watermark” and “Upload Photos” buttons, each with a label showing what’s currently loaded.

- **2. Size and opacity** — three sliders built by the shared helper `_add_slider()`: max width (10–600px), max height (10–600px), opacity (0–100%).
- **3. Position** — the 3×3 grid of `ttk.Radiobutton` widgets for the nine presets, laid out with `grid()` inside their own sub-frame, plus two fine-offset sliders ($\pm 20\%$) in a *separate* sub-frame.
- **4. Output** — a format dropdown (`ttk.OptionMenu`, JPEG/PNG), a save-folder text entry with a Browse button (`filedialog.askdirectory()`), and the green “Apply Watermark & Save All” button.

`_add_slider()` is a small reusable helper: given a label, a tkinter variable, and a min/max range, it builds a label, a `ttk.Scale`, and a live numeric readout label that updates whenever the variable’s value changes (via `var.trace_add('write', refresh)`, a callback tkinter fires whenever a tracked variable’s value is written).

Putting the offset sliders in their own sub-frame (rather than the same frame as the position radio-button grid) was not cosmetic — the Verification section of the README records that mixing `pack` and `grid` geometry managers inside one `LabelFrame` caused tkinter to raise an error during a real GUI-construction test, and the fix was exactly this separation into sub-frames.

`_build_preview()` creates two `tk.Canvas` widgets side by side (Before / After), each fixed at `PREVIEW_MAX` (300) pixels square, with a gray background and a thin border so an empty canvas is still visible before any image is loaded.

9 The full user actions

9.1 `upload_watermark()`

Opens a file picker restricted to PNG/JPEG. On success, opens the chosen file with Pillow and immediately calls `.load()` (which forces Pillow to actually read the pixel data now rather than lazily later, catching corrupt files early), stores it in `self.watermark_original` **unmodified**. The README stresses that resizing/fading always happen fresh from this original on every preview or save pass, so slider changes are **non-destructive** — the original never gets progressively resized or degraded by repeated edits.

9.2 `upload_photos()`

Opens a multi-file picker (`askopenfilenames`) restricted to JPEG/PNG, stores every selected path as a tuple in `self.photo_paths`.

9.3 `apply_and_save()`

Loops over every path in `self.photo_paths`. For each one: opens the full-resolution photo, builds a fresh watermark at the user’s requested size/opacity via `compute_watermark_box() + apply_opacity()`, computes its position with `compute_position()`, composites with `paste_watermark_at()` (the very same function the preview uses), and hands the result to `save_watermarked_file()`. Any photo that fails to open (`except Exception: continue`) is silently skipped rather than aborting the whole batch, and the final status label reports how many were actually saved.

9.4 `save_watermarked_file(image, source_path, location, file_format)`

Derives the output directory from `pathlib.Path`: either the user's chosen custom folder, or a `watermarked/` subfolder created next to the source photo. The output filename is `<original_stem>_watermarked.<ext>`. Critically: if the composited image is in RGBA mode, it is **always** saved as PNG, **regardless of the user's format dropdown choice**, because JPEG cannot store an alpha channel. This is documented as a known, intentional (if slightly surprising) limitation in the Challenges section of the README, not a crash.

10 Documented challenges (from the README) and how each was handled

- **RGBA output silently overrides the format choice.** Already described above; carried over from the original code, not something newly introduced, and explicitly flagged as "worth knowing before relying on the format dropdown" rather than hidden.
- **Live preview must stay fast regardless of source resolution.** Solved by thumbnailing to 300px before compositing anything for the preview (Section 6), so preview cost is constant no matter how large the original photos are; the final save pass still works at full resolution.
- **Keeping preview and save honest with each other.** Both call `paste_watermark_at()`; verified by running the in-process flow end to end (load synthetic images, call `_render_preview()`, then `apply_and_save()`) and confirming the saved file's existence and expected composited pixels.
- **File dialogs make full GUI automation impractical.** `askopenfilename/askopenfilenames` block on a native OS file picker, so there is no way to script a full click-through test. The workaround used was to assign `app.watermark_original` and `app.photo_paths` directly, bypassing the dialogs, to exercise everything downstream of them, while separately confirming the window itself constructs and runs under a real X display (the Linux graphical display system).

11 Testing and verification methodology

The README documents concrete, reproducible verification (not just claims), run under Python 3.12.3 and Pillow 10.1.0:

1. **Pixel-level compositing test** — the blue-base/red-watermark test from Section 5, comparing the old no-mask path against the new masked path at the pixel level.
2. **GUI construction test** — constructing a real `tk.Tk()` root and a `WatermarkApp` instance under a live X display with a 5 second timeout, confirming no exceptions on construction or on `root.update()`. This is the test that caught the `pack` vs. `grid` geometry-manager conflict mentioned in Section 10, before it could ship.
3. **Full interactive flow without native dialogs** — a synthetic 600×400 JPEG photo and a 200×100 RGBA PNG watermark are generated with Pillow, assigned directly onto the live app instance (bypassing `askopenfilename`), then `_render_preview()` and `apply_and_save()` are both called and the resulting output directory is inspected. The recorded result: `Status message: Saved 1 watermarked photo(s).` and a file named `photo_watermarked.png` (PNG because the composited RGBA result forced it, per Section 7.3's "Output controls" limitation) actually present in the output folder.

11.1 What is explicitly *not* verified

The README states plainly, and this document repeats without softening: the interactive native file-picker flow was **not** tested end to end (blocked on OS dialogs, not scriptable headlessly); drag-to-reposition on the canvas is not implemented at all (see below); and behavior on Windows is untested, though `pathlib.Path` is expected to handle it correctly by design.

12 What the author says they'd do differently

Directly from the README, unedited in substance:

- Add a real automated test suite (e.g. `unittest` cases around `compute_position()`, `apply_opacity()`, and `paste_watermark_at()`) instead of relying on ad hoc verification scripts per change.
- Support dragging the watermark directly on the preview canvas, instead of only preset positions plus offset sliders. `tkinter`'s canvas supports mouse-event binding for this, but converting canvas coordinates back to image coordinates at the preview's scale factor adds real complexity that did not feel justified for a first pass.
- Cache the opened "before" base image across preview renders instead of reopening it from disk on every slider tick; currently harmless at interactive speed but wasteful for very large source photos.

13 Glossary (quick reference)

- **Alpha channel** — the fourth number per pixel in RGBA images, encoding opacity from 0 (transparent) to 255 (opaque).
- **Alpha compositing** — blending two images by weighting their colors according to alpha, rather than overwriting one with the other.
- **Bounding box resize** — shrinking/growing an image to fit inside a maximum width/height while preserving its aspect ratio.
- **Debounce** — delaying a repeated action until rapid triggering events pause, to avoid redundant work.
- **Instance attribute** — a value stored on one specific object (here, one `WatermarkApp`), as opposed to a module-level global shared by everything.
- **Mask (Pillow)** — an image passed to `paste()` whose values control how much of the pasted image shows through at each pixel.
- **RGBA** — red/green/blue/alpha pixel color mode.
- **tkinter / ttk** — Python's built-in GUI toolkit and its themed widget variant.